

TP sécurité des communications

Ce TP aborde les fondamentaux de la sécurité des communications à travers des étapes pratiques et complémentaires, visant à construire une communication chiffrée, authentifiée et robuste face aux attaques.

Partie 1: Échange sécurisé d'une clé symétrique AES

Une entité (Alice) souhaite échanger une clé AES avec une deuxième entité (Bob), sans qu'un attaquant puisse l'intercepter.

L'approche proposée repose sur les étapes suivantes :

1. Établir un secret partagé avec l'algorithme du Diffie-Hellman

Implémentez l'algorithme Diffie-Hellman pour permettre à deux entités (Alice) et (Bob) de générer un secret partagé k .

2. Utilisation du secret partagé pour chiffrer

- Générer une **clé AES** (128 bits) chez Alice.
- Chiffrer la clé AES générée avec le secret partagé k (Le chiffrement peut être une opération **XOR simple**).

3. Vérification de l'intégrité

Avant d'envoyer la clé AES chiffrée, il est important de garantir son intégrité. Pour cela, Alice doit calculer un **hash SHA-256 de la clé AES en clair**, et le transmettre avec la clé chiffrée (dans le même message ou séparément). Bob doit effectuer la vérification de l'intégrité de la clé chiffrée reçue.

4. Confirmation de la clé par retour de message

Pour s'assurer que Bob a bien reconstitué la bonne clé AES, proposez une extension dans laquelle **Bob chiffre un message de confirmation avec la clé AES reçue**, et l'envoie à Alice.

Partie 2 : Communications chiffrée RSA (Client ↔ Serveur)

Objectif :

Cette partie vise à mettre en œuvre une communication sécurisée entre un client et un serveur, basée sur **le chiffrement asymétrique RSA**.

Comportement attendu :

Le client récupère la clé publique RSA du serveur, chiffre un message, puis l'envoie via une requête HTTP. Le serveur déchiffre ce message avec sa clé privée et renvoie la version **en clair** au client.

Mise en place :

Installez Python et les bibliothèques nécessaires : flask, requests et rsa (ou cryptography). Le serveur doit fournir deux points d'accès : l'un pour partager sa clé publique, l'autre pour recevoir les messages chiffrés. Il doit pouvoir déchiffrer ces messages à l'aide de sa clé privée. Le client, de son côté, récupère la clé publique, chiffre un message, puis l'envoie au serveur via une requête HTTP. L'ensemble du projet peut être exécuté localement sur un seul ordinateur, à l'aide de deux terminaux.

Partie 3 : Interception et modification de messages (MITMPROXY)

Objectif :

Approfondir l'analyse de la communication sécurisée mise en place dans la Partie 2, en simulant une attaque de type Man-In-The-Middle (utilisation d'un proxy). Cette étape permet d'évaluer les limites du chiffrement asymétrique lorsqu'aucun mécanisme d'authentification ou d'intégrité n'est présent.

Mise en place :

Utilisez un proxy tel que **mitmproxy** pour intercepter localement les échanges entre le client et le serveur définis dans la Partie 2. Le client doit être configuré pour passer par ce proxy. Une fois en place, vous pourrez observer et modifier les messages transmis, afin d'évaluer le comportement du serveur et/ou du client face à une requête corrompue.

- Ajoutez un attaquant qui modifie la requête du client et/ou la réponse du serveur

Que constatez vous ?

Partie 4 : Authenticité et intégrité via signature numérique

Objectif :

Compléter la sécurité de l'échange en assurant non seulement la confidentialité (Partie 2), mais aussi l'**authenticité de l'émetteur** et l'**intégrité des données**.

Pour cela, le client devra **générer sa propre paire de clés RSA** (car il n'en possède pas encore), puis **signer numériquement** le message clair avec sa clé privée avant de l'envoyer. Ensuite le client envoie au serveur : message chiffré + signature + sa clé publique. Le serveur pourra alors vérifier que le message :

- vient bien du client (authenticité du message),
- n'a pas été altéré (intégrité).
- Le serveur répond au client avec le contenu du message en clair, comme en Partie 2, **accepter ou rejeter** le message en informant explicitement le client de la validité ou non de la signature.

Test d'une attaque par usurpation d'identité avec signature valide :

Reproduisez une attaque similaire à celle de la Partie 3, mais dans le contexte du protocole avec signature numérique.

Testez les deux scénarios suivants :

1. **Remplacement de la clé publique dans la requête** : Interceptez une requête légitime et modifiez uniquement la clé publique du client (en injectant une clé tierce). Que se passe-t-il au niveau de la vérification de la signature ?
2. **Usurpation d'identité via remplacement de contenu signé**:

Objectif : Illustrer qu'une signature numérique n'est pas suffisante pour authentifier l'émetteur.

Attaque à réaliser : Laissez le client envoyer un message normalement. Interceptez cette requête puis remplacez dans cette requête le message chiffré, la signature et la clé publique du client, par celles de l'attaquant, pour un message différent.

Observez que le serveur accepte le message comme s'il venait du client et Vérifiez côté serveur que le message affiché est bien celui envoyé par l'attaquant.

Partie 5 : Messagerie sécurisée

Objectif :

Concevoir et réaliser une communication sécurisée (messagerie sécurisée) entre deux clients (Alice et Bob), dans laquelle les messages sont échangés chiffrés avec une clé de session AES, elle-même transmise de manière sécurisée via RSA par un serveur de confiance.

Scénario :

1. Chaque client possède une paire de clés RSA.
2. Au lancement, les clients s'enregistrent auprès du serveur avec leur clé publique.
3. Lorsqu'un client souhaite discuter avec un autre, il s'authentifie auprès du serveur et demande une session sécurisée.
4. Le serveur :
 - Génère une clé de session AES unique.
 - La chiffre avec la clé publique RSA de chaque client.
 - Transmet la clé chiffrée AES à chacun.
5. La communication entre les clients se fait directement et chiffrée avec AES/Fernet.

Recommandations

- Utiliser une page HTML simple pour saisir et afficher les messages.
- Afficher dans le terminal le contenu chiffré/clair transmis entre les clients.
- Les clés peuvent être stockées dans un dossier local et accessibles à tous.
- Une clé AES donnée pour une session ne doit être générée qu'une seule fois, et réutilisée ensuite.

Extension possible

- Ajout d'une signature RSA des messages envoyés.
- Possibilité de supporter plusieurs paires de clients (ou ajout d'une "nouvelle session" avec un autre client) !
- Ajout de timestamps / Invitation ou de notification de session.
- Historique local des discussions.